

Cortex-M85 X-Wing의 아키텍처 특화 공동 최적화: 단계별 동일 ELF 검증과 스티칭의 한계

익명 저자

익명 기관

anonymous@example.com

Abstract. 본 연구는 Cortex-M85의 MVE, 제한된 듀얼 이슈와 레지스터·LSU 제약에 맞춰 ML-KEM-768과 X25519 구현을 함께 조정하는 X-Wing 아키텍처 특화 구현 연구다. X-Wing draft-10 [6]의 최초 all-off A0와 최종 B8을 하나의 ELF에서 직접 비교한 별도 재플래시 5회의 보수적 개선률은 키생성 19.52%, 캡슐화 19.75%, 역캡슐화 warm 11.66%, cold 15.48%였다. B8은 NTT·저장 융합, MVE 데이터 변환, fixed-base comb, X25519 체 연산 재스케줄링, 정렬 인식 메모리 함수, MVE 조건부 교환과 두 X25519 출력의 zero-safe paired inversion을 결합한다. 단계별 동일 ELF 진단은 여섯 1단계 축의 11.22–25.96% 개선과, 그 승자 위에서 MVE 조건부 교환·paired inversion이 만드는 캡슐화 4.226–4.228%의 추가 국소 효과를 분리하지만 서로 다른 하네스의 개선률은 합산하지 않는다. 반면 ML-KEM-X25519 명령 스티칭은 평균 1.6개의 빈 스칼라 레지스터와 숨길 수 없는 재물질화 때문에 순차 실행보다 느렸다. 동일 퇴직 명령 수의 조건부 교환 재배치는 단독 커널에서 12.883% 빨라 dual-beat 중첩 가설과 일치했지만 전체 cold 경로의 채택 문턱을 넘지 못했다. 결과적으로 기여는 새 primitive가 아니라 기존 기법의 프로토콜 적용 위치, M85 자원 제약과 동일 ELF 종단 간 판정을 결합한 공동 최적화 및 적용 한계의 실증이다.

Keywords: PQ/T 하이브리드 · X-Wing · Cortex-M85 · 아키텍처 특화 구현 · 공동 최적화 · MVE · 동일 ELF 실보드 측정

1 서론

포스트양자 전환기에 고전 방식과 포스트양자 방식을 함께 사용하는 하이브리드 키 확립이 제안되어 왔다. X-Wing은 X25519와 ML-KEM-768을 독립적으로 실행하고 두 공유 비밀을 SHA3-256으로 결합한다 [4,6]. 두 구성요소는 서로 다른 연산 자원을 사용하므로 각각의 구현을 최적화할 기회가 크다. 그러나 커널 감소율, 정적 명령 수와 미점유 발행 용량은 API 전체의 실제 이득을 보장하지 않는다.

본 연구의 목표는 draft-10의 구성요소 최적화를 Cortex-M85 X-Wing에 통합하고, 같은 ELF에서 직접 비교해 실현된 누적 이득과 채택 한계를 밝히는 것이다. 분류상 본 연구는 새 X25519·ML-KEM primitive 설계와 기존 코드의 단순 포팅 사이에 놓이는 아키텍처 특화 구현 연구다. 중심 질문은 기존 기법을 M85의 MVE, 발행 구조와 레지스터 제약에 맞춰 공동 최적화했을 때 실제 X-Wing API 전체가 얼마나 빨라지고, 어느 후보가 통합 경계에서 실패하는가?이다. 판정은 외부 표의 cycle을 빼는 방식이 아니라 동일 코드·데이터 배치의 실보드 비교로 내린다.

1단계 통합 대상은 NTT·저장 경로, MVE 데이터 변환, fixed-base X25519, 체 곱셈·제곱과 메모리 함수다. 2단계는 남은 X25519 병목에 MVE 조건부 교환과 두 출력의 paired inversion을 적용한다. 별도 2024/039 래퍼의 함수 스티칭과 dual-beat 재스케줄링은 아키텍처 경계를 확인하는 후보로 평가한다. 정적 발행 여유나 국소 커널 이득이 보여도 동일 ELF 종단 간 성능이 회귀하거나 사전 문턱을 넘지 못하면 최종 묶음에 넣지 않는다.

본 연구의 기여는 다음 네 가지다.

1. **Cortex-M85 공동 최적화의 통합.** 공개 pqm4/Lenngren 계보의 알고리즘을 바꾸지 않고 ML-KEM의 NTT·데이터 변환, X25519의 fixed-base·체 연산·조건부 교환, 두 X25519 출력의 affine 변환과 공통 메모리 경로를 하나의 draft-10 구현에서 단계적으로 결합한다.
2. **프로토콜 위치를 이용한 잔여 병목 제거.** 16-word 조건부 교환을 MVE의 128-bit 네 묶음으로 처리하고, 캡슐화에 동시에 존재하는 fixed/variable-base projective 끝점의 역원 두 개를 zero-safe paired inversion 하나로 줄인다. 두 기법 자체의 발명보다 X-Wing 적용 위치와 전체 경로에서의 합성 효과를 기여 경계로 삼는다.
3. **M85 적용 가능성과 실패 경계의 규명.** 실제 X25519-NTT 병합에서 빈 레지스터 부족과 재물질화 비용이 스티칭 이득을 없애는 기전을 측정한다. 또한 같은 명령 수의 dual-beat 조건부 교환 스케줄이 단독 커널에서는 이득이지만 전체 경로 채택 문턱에는 못 미치는 지점을 분리한다.
4. **종단 간 채택과 검증 방법.** 후보를 micro-gate, 개별 전체 경로, 누적 동일 ELF 순서로 평가하고, 최초 A0와 최종 B8도 별도 동일 ELF 5회로 직접 비교한다. KAT·8-input·reject·low-order·stack·flash 검증과 leave-one-out 귀속을 결합하며, 단계가 다른 절대 cycle은 섞지 않고 실패 후보까지 원시 로그와 동결 산출물로 보존한다.

2 배경과 관련 연구

2.1 X-Wing과 종단 간 최적화 경계

X-Wing은 X25519와 ML-KEM-768을 독립적으로 실행하고 두 공유 비밀을 SHA3-256으로 결합하는 하이브리드 KEM이다 [4,6]. Cortex-M85 구현의 전체 시간은 X25519 체 연산과 fixed-base 연산, ML-KEM의 NTT·데이터 변환·Keccak, 메모리 이동과 결합 코드가 함께 결정한다. 따라서 한 커널의 명령 수 감소나 처리율만으로 API 전체 개선률을 계산할 수 없다.

기준 A는 pqm4 ML-KEM/Keccak [10], Lenngren X25519 [12], 스칼라 C 데이터 변환과 newlib-nano를 Cortex-M85에서 재빌드한 경로다. B는 암호 규격과 API를 바꾸지 않고 여섯 구현 축만 교체한다. A를 공개된 모든 M85 구현 중 최고라고 주장하지 않으며, A→B는 동일 계보 안에서 최적화 토글의 인과적 효과로만 해석한다.

2.2 Cortex-M85 구현 자원과 선행 연구

Cortex-M85는 Armv8.1-M 기반 순차 실행 코어로 MVE 벡터 명령과 제한된 듀얼이슈 조합을 제공한다 [3]. MVE는 ML-KEM 데이터 병렬성에, 스칼라 MAC·

Table 1. 가장 가까운 공개 비교점과 본 연구의 차이. “직접 비교 불가”는 장치·API·판본·배치가 달라 cycle 순위를 만들지 않았다는 뜻이다.

비교점	제공 범위	본 연구와의 관계
pqm4·Lenngren	Cortex-M4 ML-KEM/Keccak·X25519 구성요소	A의 계보; M85 draft-10 전체 API 직접 비교 불가
Fujii-Aranha· Düll 등	M-profile·소형 MCU의 상수시간 Curve25519	체 연산·cswap 선행기법; MVE/X-Wing 통합은 다름
Helium NTT· pqmx	M55/M85 계열의 MVE 격자 커널	X/Y/C8·C9의 비교점; X25519· combiner 미포함
SLOTHY 계보	제약 기반 어셈블리 재스케줄링	스케줄 방법의 비교점; 프로토콜 채택 판정은 다름
본 연구	M85에서 draft-10 A0-B8 전체 API	조사한 공개 항목 중 완전한 M85 X-Wing 기준선은 없어 내부 동일 ELF 인과 비교

캐리 명령은 X25519 체 연산에 사용된다. 그러나 레지스터 수명, APSR, LSU, 곱셈기와 MVE 저장 차단 때문에 커널 이득이 통합 경계에서 그대로 유지된다고 가정할 수 없다.

M-profile X25519 최적화는 Fujii-Aranha [8]와 Lenngren 계보가 제공한다. Cortex-M55 MVE 격자 연산 [5]과 pqmx [14]는 ML-KEM 벡터화의 비교점이다. 본 연구는 새 X25519, ML-KEM 또는 NTT 알고리즘을 제안하지 않는다. RFC 7748의 상수시간 xor-mask 조건부 교환 [11]과 마이크로컨트롤러 Curve25519 구현 [7], 여러 역원을 한 번의 역원으로 처리하는 Montgomery batch inversion [9]이 직접 선행기법이다. 본 연구의 MVE 조건부 교환과 paired inversion은 이 일반 기법의 범주에 든다. 기여는 이 계보의 구현 변환을 Cortex-M85 X-Wing에 통합하고, M85 발행·레지스터 제약과 프로토콜의 두 X25519 출력 구조에 맞춰 적용 위치를 정한 뒤 동일 ELF API 경계에서 누적 효과와 실패 조건을 함께 확인한 데 있다. 따라서 외부 최신 구현과의 단순 속도 경쟁이나 새 암호 primitive의 신규성을 주장하지 않는다.

가장 가까운 공개 비교점은 세 층위다. pqm4·Lenngren과 M-profile X25519 계보 [10,12,8]는 A와 X01/X02의 구성요소를, Helium NTT·pqmx [5,14]는 X/Y/C8·C9의 MVE 격자 경로를, SLOTHY 계보 [1,2]는 재스케줄링 방법을 제공한다. 그러나 이들은 장치·API·프로토콜 판본·컴파일러·메모리 배치가 달라 M85 draft-10 전체 API의 직접 경쟁 cycle이 아니다. 따라서 외부 값은 위치 설명에만 사용하고, 성능 효과와 순위는 동일 ELF의 A-B 안에서만 판정한다.

3 아키텍처 특화 공동 최적화 설계

3.1 1단계: 동일 규격 안의 여섯 독립 축

all-off A와 all-on B는 draft-10의 combiner 순서, packed 32B secret key, 64B encapsulation seed와 FIPS 203 encapsulation-key check를 공유한다. B에서 켜는

Table 2. all-on B의 여섯 최적화 축과 적용 경계.
직접 기여는 표 10에서 같은 실행의 분모로 비교한다.

축	A→B 변환	제거한 비용과 적용 경계
X/Y/C8	기존 NTT/invNTT/store → NTT 경로 비용과 forward 뒤 scalar rev4; pqmx M85와 current-order 저장	ML-KEM 다항식 경로
C9	11개 scalar 변환 → MVE v1	packing·CBD·message 변환의 scalar 데이터 병렬 비용; byte 삽입은 scalar 유지
X06	basepoint ladder → $w = 3$ signed comb	반복 ladder, 조건부 negate와 제곱 사슬의 중간 저장·적재; fixed-base 호출만 적용
X01	원본 체 곱셈 → 고정 프레임·재스케줄	SP 갱신 의존과 명령 대기; X25519 field multiplication
X02	원본 체 제곱 → 재스케줄	명령 수를 늘리지 않은 의존 간격 조정; X25519 field squaring
K31	newlib-nano → 정렬 인식 워드 루프	고정 크기 buffer의 byte 단위 복사·초기화; 정렬·길이에만 분기

여섯 독립 축은 표 2에 정리했다. X/Y/C8은 하나의 NTT·저장 토글이고, K31은 memcpy와 memset을 함께 전환하는 하나의 토글이다.

3.2 ML-KEM: NTT·저장 융합과 데이터 변환 MVE화

X/Y는 pqmx의 Cortex-M85 forward/inverse NTT를 기존 Plantard 경로의 domain 계약에 맞춰 연결한다. inverse 입력의 centered 범위 $[-1664, 1664]$ 를 보드에 확인하고, host residue oracle와 8-seed full-byte 동치를 통과한 factor만 사용했다. C8은 consumer마다 scalar rev4를 수행하지 않고 pqmx의 current-order MVE structure-store를 재사용해 forward가 필요한 순서를 직접 내도록 한다. basemul, matrix accumulation과 serialization 계약은 바꾸지 않는다.

C9는 CBD, 12-bit serialization, 10/4-bit compression·decompression, 두 비교 함수와 message 변환을 포함한 11개 scalar C 함수를 arm_mve.h 기반 고정 반복 구현으로 교체한다. 양자화는 MVE로 처리하되 v1에서 byte 삽입은 scalar로 남긴다. 탐색 단계의 단일축 ELF는 채택 방향을 정하는 데만 사용했다. 서로 다른 ELF의 비율을 더하지 않고, 최종 효과와 직접 기여는 각각 표 6과 표 10의 동결 ELF에서 같은 분모로 다시 잰다.

3.3 X25519와 공통 메모리 경로

X06은 keygen의 pk_X 와 encaps의 ct_X 두 fixed-base 호출에서 basepoint 9를 일반 Montgomery ladder로 처리하는 대신 $w = 3$ signed comb와 33,024 B flash 표를 사용한다. 선택 부호는 field negate 대신 z_3/t_3 포인터를 상수시간으로 맞바꾸고, 254회 제곱 사슬은 중간 값을 레지스터에 유지해 매 호출의 store/load를 제거한

다. variable-base 호출과 warm 역캡슐화에는 적용 지점이 없으며, 표 10의 warm 16 cycle은 검출 가능한 기여가 아니라 측정잡음 규모로 해석한다.

X01은 fe25519_mul의 push/pop 기반 프레임을 고정 SP 오프셋으로 평탄화한 뒤 명령을 재배치하고, X02는 fe25519_sqr의 채택 심볼을 94명령 그대로 재배치한다. K31은 주소 정렬과 길이에만 분기하는 word memcpy/memset을 사용한다. 후보 심볼은 두 함수 합계 264B이며, 컴파일러가 인라인한 복사는 교체·계수 범위 밖이다.

3.4 2단계: MVE 조건부 교환과 두 출력의 paired inversion

1단계 누적 구현 뒤 남은 X25519 비용에는 두 변환을 적용했다. 첫째, variable-base Montgomery ladder의 16개 32-bit word 조건부 교환을 scalar xor-mask 루프에서 네 개의 128-bit MVE load/xor/and/xor/store 묶음으로 바꿨다. 마스크가 0 또는 0xffffffff인 고정 명령 경로를 유지하며, 다른 알고리즘의 상태를 동시에 살려 두지 않으므로 스티칭과 달리 추가 스칼라 레지스터 분할이 필요하지 않다.

둘째, 캡슐화에는 같은 ephemeral scalar로 만드는 fixed-base ciphertext와 variable-base 공유 비밀의 projective 끝점 두 개가 동시에 존재한다. 각 분모를 따로 역원화하는 대신, 0이 아닌 분모 d_1, d_2 에 대해 $t = (d_1 d_2)^{-1}$ 를 한 번 계산하고 $d_1^{-1} = d_2 t$, $d_2^{-1} = d_1 t$ 로 복원한다. 0 분모는 상수시간 canonical zero test 뒤 1로 대체하고 마지막 출력을 마스크해 기존 low-order/all-zero 동작을 보존한다. 역캡슐화에는 X25519 출력이 하나뿐이므로 이 변환을 적용하지 않는다. 이 두 축은 알려진 연산 기법의 새로운 암호학적 primitive가 아니라, X-Wing의 데이터 흐름과 M85 실행 자원에 맞춘 구현 변환이다.

4 평가 방법

4.1 기준선, 장치와 반복 단위

주 비교의 A/B는 같은 draft-10 입력, API, 동결 ELF와 링크 배치를 공유한다. 런타임 디스패처가 두 경로에 공통으로 있으므로 $A \rightarrow B$ 대응효과만 성능 개선률로 사용하고, 이 ELF의 절대 cycle을 디스패처 없는 배포형 값으로 해석하지 않는다. 판본·API·컴파일러·배치가 다른 외부 cycle도 분모로 쓰지 않는다.

역캡슐화 warm은 2,464B로 확장한 decapsulation key를 재사용하는 경로이고, cold는 packed 32B seed에서 expandDecapsulationKey를 수행한 뒤 역캡슐화한다. cold와 warm은 다른 workload이며 하나의 latency로 합치지 않는다. 디스패처 없는 all-on B의 별도 절대값은 키생성 627,069, 캡슐화 988,456, warm 780,020, cold 1,406,887 cycle이다. 이 값은 같은-ELF 상대효과의 분모와 섞지 않는다.

장치는 EK-RA8M1(Cortex-M85, 480 MHz), 컴파일러는 GCC 13.2.1 -O2다. 코드는 ITCM, 데이터와 stack은 DTCM에 두고 DWT CYCCNT에서 빈 하네스 25 cycle을 차감한다. 각 셀은 100회 중앙값이고, 별도 재플래시가 반복 단위다. 실험별 반복과 효과 수는 표 3에 고정한다.

Table 3. 주 결과와 귀속 실험의 독립 반복 단위. 셀 안의 $N = 100$ 은 중앙값 계산용이다.

실험·용도	독립 반복과 순서	본문 역할
DJ 최종 A0/B8	재플래시 5회; A0-B8-B8-A0; 연산별 10 대응효과	최초 대 최종 headline
CY 1단계 A/B	재플래시 5회; ABBA; 연산별 10 대응효과	여섯 측의 단계 효과
DH 2단계 경로	재플래시 2회; 후보 순서 회전; 셀당 100회	cswap·paired inversion 귀속
CU leave-one-out	재플래시 6회; 측·모드당 네 실행 내 대비	측별 기여와 상호작용

Table 4. 보조 진단의 독립 반복과 사용 범위.

실험	반복과 사용 범위
DF·DI·DD	각 재플래시 2회; micro 채택 gate, dual-beat 가설, DWT-off PMU 진단
CW·CX	8-input 재플래시 6회; timing 유효 재플래시 2회, 클래스당 3,000 표본·24 t 값
CV+CZ	재플래시 7회, 연산당 14효과; 스티칭 후보 기각에만 사용
DE	A/B 독립 링크 각 2회, 진단 ELF 각 1회; footprint·stack

CU와 CW의 네 대비는 한 재플래시 안의 두 A와 두 B 셀 중앙값의 모든 조합이며 독립 표본이 아니다. 하드웨어 반복 단위는 표의 재플래시 수다.

이 보드 개체의 ITCM 0x18과 0xE0에서 각각 8-byte 쓰기 고착 단위를 관찰했다. itcm_pad.c의 256-byte 링크 패드로 모든 측정 코드를 0x100 이후에 배치해 회피했고, 플래시 읽기 검증과 각 실행 전 KAT를 통과한 경우만 채택했다. 정상 ITCM을 가진 두 번째 보드 반복은 미측정이다.

4.2 정확성, 입력과 상수 시간 검증 범위

정확성 검사는 RFC 7748 KAT [11], SHA3-256 KAT, pqm4 위의 ML-KEM-768 왕복·암묵적 거부 [13]와 8-seed A/B 전 바이트 비교를 포함한다. 별도 KAT ELF에서는 draft-10 Appendix C vector 1을 자동 파싱해 public key 1,216 B, ciphertext 1,120 B, shared secret 32 B와 역캡슐화 결과 전량을 보드 출력과 대조했다. 그 ELF는 배열 추가로 성능 ELF보다 text가 2,144 B 커서 cycle은 폐기하고 정확성 게이트로만 사용했다.

주 B에는 고정/무작위 ciphertext 타이밍 검정을 수행했다. 두 클래스는 같은 버퍼와 측정 전 memcpy를 사용한다. 클래스당 3,000 표본, 별도 재플래시 2회와 p100/p90/p75/p50 절단의 24개 Welch t 값이 모두 $|t| < 2.0$ 이었고, B warm/cold의 최댓값은 1.99/1.49였다. 이는 해당 입력 분류에서 누설을 검출하지 못했다는 결과일 뿐, 비밀키 의존 누설이나 캐시·전력 채널의 부재를 증명하지 않는다.

2단계 최종형은 두 독립 플래시에서 RFC 7748/SHA3 KAT, 8개 결정적 draft-10 입력의 pk/sk/ct/enc/warm/cold/reject 전 바이트 동치, warm/cold 일치와

low-order 입력 3개를 통과했고 `harness_fails=0`이었다. 이 검증은 paired inversion의 0 분모 처리와 MVE 조건부 교환이 기존 API 출력을 바꾸지 않았는지 확인하는 2단계 채택 게이트다. 최초 A0과 최종 B8을 직접 비교한 DJ도 같은 검사를 별도 재플래시 5회 반복했고, 모든 실행에서 저차점·거부·stack guard mismatch와 `harness_fails`가 0이었다.

5 종단 간 성능 결과

5.1 최초 A0 대 최종 B8의 직접 동일 ELF 결과

표 5이 본 연구의 주 성능 결과다. A0은 여섯 1단계 측과 두 2단계 변환을 모두 끄고, B8은 최종 채택한 여덟 변환을 사용한다. 하나의 ELF에서 A0-B8-B8-A0 순서를 재플래시 5회 반복해 인접한 AB/BA의 10개 대응효과를 얻었다.

Table 5. draft-10 최초 A0 대 최종 B8의 직접 동일 ELF 결과. 각 셀은 100회 중앙값이며 개선률 범위는 5회 재플래시의 인접 AB/BA 10개 대응효과다.

연산	A0 범위 (cycle)	B8 범위 (cycle)	개선률 범위	보수적 값
키생성	870,869–870,970	700,834–700,865	19.521–19.534%	19.52%
캡슐화	1,326,924–1,326,931	1,064,853–1,064,862	19.750–19.751%	19.75%
역캡슐화 warm	934,803–934,837	825,746–825,793	11.661–11.670%	11.66%
역캡슐화 cold	1,802,626–1,802,655	1,523,463–1,523,502	15.484–15.488%	15.48%

다섯 실행은 동일 ELF/SREC와 flash readback을 사용했고 정확성 gate를 모두 통과했다. 이 결과에는 양쪽에 공통인 실험용 mode 조회와 두 경로 코드 배치가 포함되므로 절대 cycle을 dispatcher-free 배포값으로 해석하지 않는다. 대신 같은 조건의 A0-B8 차이는 최초에서 최종까지의 직접 효과다. 뒤의 단계별 표는 변환 귀속을 위한 별도 진단이며 이 주 결과에 더하거나 곱하지 않는다.

5.2 1단계 여섯 측의 동일 ELF 귀속

표 6은 최종 결과를 여섯 1단계 측에서 분해하기 위한 단계 진단이다. A와 B는 같은 draft-10 입력, API, 동결 ELF와 링크 배치를 공유하고 여섯 최적화 토글만 다르다. 별도 재플래시 5회의 AB/BA 두 대응효과, 즉 모드별 10개 값의 전체 범위와 최솟값의 두 자리 반올림을 보고한다.

Table 6. draft-10 동일 동결 ELF의 누적 all-off A→all-on B 효과. 범위는 별도 재플래시 5회, ABBA, 셀당 $N = 100$ 중앙값에서 얻은 모드별 10개 대응값이다.

모드	A 범위 (cycle)	B 범위 (cycle)	개선률 범위	최솟값
키생성	870,370–870,444	644,356–644,390	25.964–25.972%	25.96%
캡슐화	1,321,221–1,321,241	1,053,681–1,053,699	20.248–20.250%	20.25%
역캡슐화 warm	934,139–934,155	829,299–829,315	11.222–11.224%	11.22%
역캡슐화 cold	1,801,432–1,801,449	1,471,797–1,471,828	18.297–18.299%	18.30%

다섯 실행 모두 KAT, `harness_fails=0`, 8개 중값의 pk/sk/ct/ss와 암묵적 거부 바이트 동치, stack canary와 동일 flash readback을 통과했다. 개선률 범위 폭은 0.002–0.009%p로 사전등록한 0.05%p보다 작았다. 네 모드의 단순평균은 18.93%지만 warm과 cold는 다른 workload이므로 평균을 대표 성능으로 사용하지 않고 연산별 값과 11.22–25.96%의 최솟값 범위를 보고한다.

5.3 2단계 잔여 병목의 전체 경로 효과

표 7은 1단계 누적 승자를 새 동일 ELF의 기준 A로 두고 MVE 조건부 교환(C), paired inversion(B), 둘의 결합(D)을 비교한 결과다. 각 값은 별도 플래시 한 번에서 100회 측정된 중앙값이며 표에는 두 플래시의 범위를 보인다.

Table 7. 2단계 동일 ELF 캡슐화 결과. 개선률은 각 행 A에 대한 두 독립 플래시의 대응값 범위이며, 1단계 절대 cycle과 섞지 않는다.

모드	A 범위 (cycle)	후보 범위 (cycle)	A 대비 개선
MVE cswap	1,128,441–1,128,448	1,114,176–1,114,177	1.264–1.265%
paired inversion	1,128,441–1,128,448	1,095,069–1,095,076	2.957%
둘 다	1,128,441–1,128,448	1,080,740–1,080,749	4.226–4.228%

Table 8. 2단계 MVE cswap의 역캡슐화 결과.

모드	A 범위 (cycle)	MVE 범위 (cycle)	A 대비 개선
warm	839,319–839,328	824,987–824,991	1.707–1.709%
cold	1,553,257–1,553,270	1,538,838–1,538,852	0.928%

결합 D는 paired inversion 단독보다도 1.308–1.309% 빨라 두 변경이 같은 병목을 단순 대체하지 않고 합성됨을 보였다. keygen은 두 변환의 적용 지점이 없어 변하지 않는다. 2단계 ELF에는 모든 모드에 공통인 실험용 scalar/MVE 조치가 추가되어 1단계와 절대 cycle이 다르다. 따라서 두 단계의 개선률을 곱한 수치는 설명용 추정치 될 수는 있어도, 본 논문의 단일 동일 ELF 실측 결과로 주장하지 않는다.

5.4 PMU 퇴직 명령 수 보조 진단

DWT cycle counter가 PMU 이벤트 계수와 충돌하는 구성을 분리하기 위해 DWT를 끈 별도 ELF를 두 번 독립 플래시했다. software increment 100회와 100,000 NOP의 16-bit 연결 계수기를 먼저 검증하고, A–B–B–A의 cycle과 INST_RETIRED를 쌍으로 계수했다. 두 실행의 모드당 네 값에서 instruction은 셀 간 동일했다.

Table 9. DWT-off PMU 보조 진단. cycle은 모드당 네 값의 중앙값이며 instruction은 동일한 네 값이다. 이 cycle은 표 6의 DWT 결과와 합치지 않는다.

연산	A cycle	B cycle	A instruction	B instruction (감소)
키생성	865,820	659,932	990,042	721,891 (27.08%)
캡슐화	1,317,054	1,069,764	1,408,965	1,132,096 (19.65%)
역캡슐화 warm	929,823	828,913	1,045,841	949,308 (9.23%)
역캡슐화 cold	1,792,330	1,487,465	2,030,943	1,670,099 (17.77%)

따라서 주 DWT 결과의 방향은 PMU cycle에서도 유지됐고, 개선은 네 연산 모두에서 퇴직 명령 감소를 동반했다. 반면 STALL_OP과 front/back-end 세 이벤트는 모든 셀에서 0이어서 stall이 없다는 뜻으로 해석하지 않고 지원·구성 미확인으로 남긴다.

5.5 개별 기여와 상호작용

leave-one-out은 B에서 축 i 만 끈 B_{-i} 를 같은 ELF에서 비교한다. 축별 $B_{-i} - B$ 는 여섯 실행에 있고, 같은 실행의 A-B 셀은 run 2-6에 있다. 표 10은 셀·합·잔차의 항등식을 보존하기 위해 A-B를 포함한 첫 완전 실행인 run 2를 한 예로 보인다. 유리한 축별 셀을 실행 사이에서 골라 합한 값이 아니며, 전 실행 범위는 표 뒤에 별도로 보고한다.

Table 10. leave-one-out의 첫 완전 실행(run 2)에서 계산한 $B_{-i} - B$ 직접 기여와 같은 실행의 가법 분해. 단위는 cycle이며 열 순서는 keygen/encaps/warm/cold다.

축	keygen	encaps	warm	cold
X06 fixed-base	152,860	152,710	16	152,732
X/Y/C8 NTT/store	22,556	18,002	28,180	50,808
C9 MVE 변환	11,120	33,300	39,386	50,552
K31 memcpy/memset	12,660	10,042	9,906	21,380
X02 체 제공	3,488	22,770	19,296	22,794
X01 체 곱셈	4,182	11,950	7,632	11,898
합	206,866	248,774	104,416	310,164
같은 ELF A-B	226,096	268,196	104,717	329,445
상호작용 잔차	19,230 (8.5%)	19,422 (7.2%)	301 (0.3%)	19,281 (5.9%)

축별 기여의 여섯 재플래시 간 최대 폭은 X06 keygen의 1,058 cycle이다. 실행 간 범위는 표 11와 같다.

Table 11. keygen · encaps leave-one-out 실행 간 범위.

기여 합은 run 1–6, A–B · 잔차 · 설명 비율은 A–B 셀을 함께 가진 run 2–6의 범위다.

항목	keygen	encaps
기여 합 (cycle)	205,927–207,076	248,549–249,616
A–B (cycle)	225,299–226,330	267,996–268,465
잔차 (cycle)	19,230–19,410	18,849–19,508
설명 비율 (%)	91.39–91.49	92.72–92.98

Table 12. warm · cold leave-one-out 실행 간 범위. 반복 범위의 정의는 표 11와 같다.

항목	warm	cold
기여 합 (cycle)	104,344–105,098	309,992–311,334
A–B (cycle)	104,717–105,190	329,268–330,231
잔차 (cycle)	1–301	18,853–19,281
설명 비율 (%)	99.71–99.999	94.15–94.29

fixed-base가 관여하는 세 모드의 잔차는 약 19k cycle로 유사하지만 2축 동시 off를 재지 않았으므로 어느 상호작용 쌍에도 인과적으로 배분하지 않는다. 같은 실행의 A–B를 분모로 한 X06 비중은 keygen 67.37–67.61%, encaps 56.58–56.94%, cold 46.11–46.36%다. X06은 세 모드에서 가장 큰 단일 축이지만 일괄적으로 “3분의 2”는 아니다. 적용 지점이 없는 warm의 X06은 $-56+16$ cycle로 잡음 규모였고, 나머지 다섯 축은 통합 후 모든 실행에서 양의 기여를 유지했다.

5.6 입력 강건성과 자원비용

8개 (sk, ct)를 반복마다 바꾼 별도 재플래시 6회의 모드당 24개 실행 내 셀 대비에서 warm은 12.97–13.02%, cold는 18.58–18.62% 개선됐다. 표 6과 부호와 크기는 일치하지만 buffer와 cache가 RAM에 있고 배치도 달라 수치를 합치거나 차이를 특정 원인에 귀속하지 않는다.

Table 13. 누적 최적화의 확인 자원비용. expCJ 결합 ELF와 dispatcher-free 최소 배포 프로록시를 구분한다.

제보·대상	확인값과 해석 경계
expCJ 결합 ELF	text/data/bss 846,028/7,572/393,740 B, stack 11,560 B; A/B 별 증분이 아님
A 최소 프로록시	text/data/bss 30,756/4/1,470 B; 비휘발 30,760 B, 정적 RAM 1,474 B
B 최소 프로록시	text/data/bss 63,028/2,040/1,474 B; 비휘발 65,068 B, 정적 RAM 3,514 B
최소 B-A	비휘발 +34,308 B (+111.53%), 정적 RAM +2,040 B (+138.40%)
X06	flash 표 33,024 B; 최소 비휘발 증가의 96.25%; 표의 ITCM 증분 0
K31	계측을 제외한 후보 wrapper 두 심볼 $132 \times 2 = 264$ B
1단계 연산별	A와 B 모두 keygen/encaps/warm/cold
stack	9,176/6,664/6,600/10,264 B; 64 B guard 훼손 0
최종 직접 비교	A0와 B8 모두 10,360/7,848/7,784/11,448 B; guard 훼손 0. 공통
stack	비교 wrapper 포함
2단계 연결 심볼	X25519 본체 증가와 retained helper 합계 922 B, 모드 상태 4 B;
상계	배포형 순증가의 정확한 값은 아님

따라서 11.22–25.96%는 명확한 시간-공간 교환을 포함한다. B의 비휘발 증가는 34,308 B이며 그중 33,024 B가 fixed-base 표다. 두 독립 링크는 런타임 디스패처와 반대 경로 핵심 심볼을 제거했고 모드별 2회 ELF 해시가 일치했다. 다만 이 절대값은 동일 BSP와 결정적 시험 RNG를 포함한 배포 프로록시이지 순수 라이브러리 크기가 아니므로, 다른 플랫폼의 수치와 정규화한 Pareto 순위는 주장하지 않는다.

X06의 시간-공간 교환을 분리하면 표 14와 같다. X06을 끄면 33,024 B 표를 제거할 수 있지만 warm을 제외한 세 모드에서 약 152k cycle을 다시 낸다. 이는 1단계 B와 B_{X06} 의 비교이며, paired inversion은 fixed-base가 만든 두 점의 동시 변환을 사용하므로 X06을 뺀 최종 B8 대안의 성능으로 해석하지 않는다.

Table 14. 1단계 X06 fixed-base의 flash-cycle 선택지. cycle은 X06을 끌 때의 증가 범위이며 warm의 범위는 측정 잡음과 같은 크기다.

선택	flash 변화	B 대비 cycle 변화
X06 유지	기준	0
X06 제거: keygen	-33,024 B	+151,802–+152,860
X06 제거: encaps	-33,024 B	+151,676–+152,710
X06 제거: warm	-33,024 B	-56–+16
X06 제거: cold	-33,024 B	+151,838–+152,732

6 최적화 채택 원칙과 보조 후보

6.1 통합 뒤 다시 재는 채택 규칙

효과는 자기 분모 안에서만 정의하며 프로토콜·구현·배치가 다른 절대값은 직접 효과로 쓰지 않는다. 구성요소 이득은 통합 뒤 같은 ELF에서 다시 재고, 개별 측이 양수여도 전체 효과와의 상호작용 잔차를 보고한다. KAT·byte 동치에 실패한 후보의 cycle은 폐기하며, 입력·공간·보드의 미측정 범위까지 일반화하지 않는다.

표 15는 같은 규칙이 최신 M85 후보를 어떻게 걸러냈는지 보인다. micro 결과만으로는 최종 채택하지 않으며, 전체 경로에서 적용되는 모드가 모두 개선될 때만 유지한다.

Table 15. 아키텍처 특화 후보의 micro 결과와 최종 판정. 각 micro 값은 별도 플래시 2회에서 동일했다.

후보	micro A→B	전체 경로 관측	판정
MVE cswap	26,742→14,213; 46.851% 감소	캡슐화 1.264–1.265%, warm 1.707–1.709%, cold 0.928% 감소	채택
paired inversion	63,845→33,527; 47.487% 감소	캡슐화 2.957%; cswap과 결합 4.226–4.228% 감소	채택
direct-64 Keccak	6,402→29,228; 356.545% 증가	1,332 B 전개와 spill로 micro gate 실패	제외
MVE rejection/MAC	719→1,089; 51.460% 증가	predicate·두 128-bit spill·scalar compaction 비용	제외
dual-beat cswap 스케줄	63,555→55,367; 12.883% 감소	캡슐화/warm/cold 0.235/0.309/0.162–0.163% 감소	제외

6.2 스티칭과 dual-beat 스케줄의 적용 한계

별도 2024/039 래퍼의 순차 실행 F와 교차 배치 U를 비교했다 [4,1,2]. 같은 F 작업량의 재플래시 7회·연산별 14개 대응효과에서 U는 세 연산 모두 1.878–3.719% 느렸다. 배치 범위를 실제 Keccak 순열의 33/43(76.7%)까지 넓힌 후보도 F보다 10.969–10.975% 느렸고, 옮긴 작업은 융합 구간에서 24.50–24.54% 더 비쌌다. 따라서 낮은 coverage 하나만으로 실패를 설명할 수 없다.

기전 분리는 실제 X25519 제공 커널에 NTT layer를 삽입해 수행했다. 두 커널의 순차 합은 280.25 cycle이었지만 정확하게 동작하는 병합본은 318.12 cycle이었다. 호스트에서 동시에 빈 스칼라 레지스터는 평균 1.6개뿐인데 게스트는 3–4개 값을 요구하여 55–75개의 재물질화가 필요했다. movw/movt는 발행 슬롯을 통째로 쓰고, 대체 1dr는 MVE 메모리 연산과 겹치지 않아 숨긴 작업보다 비용이 커졌다. 이는 “레지스터가 부족하다”를 병합 실패의 측정 가능한 조건으로 구체화한다.

반대로 조건부 교환 내부에서는 같은 명령 33개를 유지하면서 sequential VPSEL의 호출당 62 cycle을 M85 overlap 순서의 54 cycle로 줄였다. 동일 명령 수에서 8 cycle이 줄어든 결과는 issue group·dual-beat 중첩 가설과 일치하지만, 사용할 수 있는 stall 이벤트가 없으므로 그 원인을 유일하게 입증하지는 않는다.

1,024-call 측정에서는 12.883%의 이득이지만 전체 캡슐화/warm/cold 감소는 0.235/0.309/0.162-0.163%였다. 사전 규칙상 cold의 0.20% 문턱을 넘지 못했으므로 이 스케줄은 최종형에 넣지 않았다. 두 음성 결과는 주 개선과 합산하지 않으며, 다른 레지스터 표현이나 다른 스티칭 쌍까지 기각하지 않는다.

7 한계와 재현성

7.1 한계

- **관본과 기준선.** 주 A→B는 draft-10의 내부 기준선이다. A는 공개 pqm4/Lenngren 계보를 같은 M85에서 재빌드했다. 표 1의 공개 자료 조사에서 완결된 M85 X-Wing 중단 간 기준선은 찾지 못했지만, 이는 부재의 증거가 아니다. 공개된 모든 M85 구현 중 최고라는 증거도 없으므로 외부 최신 구현 대비 우월성을 주장하지 않는다.
- **자원비용.** A/B 독립 최소 배포 프로시와 stack은 측정했으나 절대값에 EK-RA8M1 BSP, 1,024B main stack과 결정적 시험 RNG가 공통 포함된다. 순수 라이브러리 크기, 다른 링커·RNG·메모리 배치의 footprint는 직접 측정하지 않았다. 최종 직접 비교에서 경로별 stack과 2단계 retained 심볼 922B·상태 4B는 확인했지만, 공통 비교 wrapper를 제거한 B8의 정확한 dispatcher-free footprint와 stack은 별도로 다시 측정하지 않았다.
- **기여 귀속.** leave-one-out 합과 전체 효과 사이 0.001-8.6% 잔차는 직접 측정했지만 2축 동시 off를 재지 않아 개별 축에 배분하지 않는다.
- **반복과 보드.** 표 3의 수치는 ITCM 고착 단위를 패드로 회피한 한 EK-RA8M1에서 얻었다. 최종 A0-B8은 다섯 독립 재플래시를 거쳤지만, 2단계 분리 실험은 두 번이다. 물리 전원 cycle, 정상 두 번째 보드, 다른 M85 SoC, Cortex-M55/M52와 전력·에너지는 미측정이다.
- **상수 시간.** 고정/무작위 ciphertext 검정은 클래스당 3,000표본에서 누설을 검출하지 못했지만 비밀키 의존 누설, cache·전력·전자기 채널은 미측정이다.
- **PMU와 보조 후보.** 퇴직 명령 수는 측정했지만 PMU 발행 class별 stall 이벤트는 0이어서 지원·구성이 미확인이다. 동일 명령 수의 cycle 차이는 dual-beat 중첩 가설과 일치할 뿐 인과 증명은 아니다. alias-safe 전 조각 solver는 미측정이며, 스티칭 기각은 평가한 field 표현·batch·통합 방식에만 적용된다.
- **단계 간 비교.** 1단계와 2단계는 각각 내부적으로 동일 ELF지만 서로의 하네스·디스패치가 다르다. expDJ가 초기 A0와 최종 B8의 직접 효과를 제공하므로 이를 주 총계로 사용한다. 1·2단계의 별도 개선률은 귀속 진단이며 서로 곱하거나 다른 절대 cycle의 차이를 총효과로 사용하지 않는다.

7.2 재현 아티팩트

재현 자료는 단계별 동결 디렉터리에 보존한다. 1단계 누적·귀속 자료는 artifacts/2026-08-27_expCJ와 CU-DE 계보에, 2단계 micro·전체 경로 자료는 artifacts/2026-08-31_expDF_DH에, dual-beat 스케줄 자료는 artifacts/2026-08-31_expDI에, 최종 A0-B8 직접 비교는 artifacts/2026-08-31_expDJ에 각각 사전등록, 원시 로그, 소스, ELF/SREC 해시와 결과 문서를 보존한다. artifact/README.md는 ITCM 결합과 itcm_pad.c 회피를 기록하며, 제출 패키지는 이 동결 자료와 정본 PDF의 해시를 함께 검증해야 한다.

8 결론

본 연구는 새 암호 primitive의 제안도 기존 코드의 단순 포팅도 아닌, Cortex-M85 X-Wing의 아키텍처 특화 구현 연구다. 초기 A0와 최종 B8을 하나의 ELF에서 직접 비교한 다섯 독립 재플래시에서 키생성·캡슐화·역캡슐화 warm/cold cycle을 각각 최소 19.52/19.75/11.66/15.48% 줄였다. 이것이 본 논문의 주 종단 간 결과다.

귀속 진단인 1단계 동일 ELF에서는 여섯 기존 구현 축을 공동 최적화하여 같은 네 연산을 25.96/20.25/11.22/18.30% 줄였다. leave-one-out 합은 전체 효과의 91.4–99.999%를 설명했고, X06 fixed-base가 관련 세 연산 감소의 46.1–67.6%로 가장 큰 단일 축이었다. 그 대가의 대부분은 33,024 B 표였으며, 표 제거 시 세 관련 모드에서 약 152k cycle이 되돌아오는 시간–공간 선택지를 정량화했다.

2단계에서는 MVE cswap과 zero-safe paired inversion만 종단 간 게이트를 통과했다. 첫 단계 누적 구현을 기준으로 둘의 결합은 캡슐화를 4.226–4.228% 더 줄였고, MVE cswap은 warm/cold를 1.707–1.709%/0.928% 더 줄였다. 이 기여는 기법 자체의 발명보다 X-Wing의 두 X25519 출력과 M85 실행 자원에 맞춘 적용, 합성 및 검증에 있다. 두 단계의 하네스가 달라 별도 개선률을 합치지 않고, 위 A0–B8 직접 비교만 총계로 사용한다.

실제 ML-KEM–X25519 스티칭은 평균 1.6개의 빈 레지스터와 숨길 수 없는 재물질화 때문에 순차 실행보다 느렸다. dual-beat 인식 cswap은 단독 커널에서 12.883% 빨랐지만 전체 cold 경로의 사전 문턱을 넘지 못했으며, 그 원인은 가설과 일치할 뿐 인과적으로 확정하지 않았다. 이 성공·실패 경계와 micro에서 전체 API까지 이어지는 채택 절차가 통합 이외의 핵심 기여다. 결론은 평가한 구현과 한 EK-RA8M1 개체에 한정하며, 외부 최신 구현 대비 우월성, 새 암호 알고리즘의 신규성, 에너지 효율 또는 보편적 시간–공간 우월성을 주장하지 않는다.

Disclosure of Interests. 저자들은 본 논문의 내용과 관련하여 공개할 경쟁적 이해관계가 없음을 선언한다.

References

1. Abdulrahman, A., Becker, H., Kannwischer, M.J., Klein, F.: Fast and clean: Auditable high-performance assembly via constraint solving. *IACR Transactions on Cryptographic Hardware and Embedded Systems* **2024**(1), 87–132 (2024). <https://doi.org/10.46586/tches.v2024.i1.87-132>
2. Abdulrahman, A., Kannwischer, M.J., Lim, T.H.: Enabling microarchitectural agility: Taking ml-kem and ml-dsa from cortex-m4 to m7 with slothy. In: *Proceedings of the 20th ACM Asia Conference on Computer and Communications Security*. pp. 1756–1771. ACM (2025). <https://doi.org/10.1145/3708821.3736210>
3. Arm Limited: Arm cortex-m85 processor software optimization guide. Tech. Rep. 107950_0100_01_en, Arm Limited (2023), issue 01
4. Barbosa, M., Connolly, D., Duarte, J.D., Kaiser, A., Schwabe, P., Varner, K., Westerbaan, B.: X-wing: The hybrid kem you’ve been looking for. *IACR Communications in Cryptology* **1**(1) (2024). <https://doi.org/10.62056/a3qj89n4e>
5. Becker, H., Bermudo Mera, J.M., Karmakar, A., Yiu, J., Verbauwhede, I.: Polynomial multiplication on embedded vector architectures. *IACR Transactions on Cryptographic Hardware and Embedded Systems* **2022**(1), 482–505 (2022). <https://doi.org/10.46586/tches.v2022.i1.482-505>
6. Connolly, D., Schwabe, P., Westerbaan, B.E.: X-wing: General-purpose hybrid post-quantum kem. Internet-Draft **draft-connolly-cfrg-xwing-kem-10** (2026), work in Progress, 2 March 2026
7. D’ull, M., Haase, B., Hinterw’alder, G., Hutter, M., Paar, C., Sánchez, A.H., Schwabe, P.: High-speed curve25519 on 8-bit, 16-bit, and 32-bit microcontrollers.

- Designs, Codes and Cryptography **77**(2–3), 493–514 (2015). <https://doi.org/10.1007/s10623-015-0087-1>
8. Fujii, H., Aranha, D.F.: Curve25519 for the Cortex-M4 and beyond. In: Progress in Cryptology – LATINCRYPT 2017. LNCS, vol. 11368, pp. 109–127. Springer (2019). https://doi.org/10.1007/978-3-030-25283-0_6
 9. Hankerson, D., Menezes, A.J., Vanstone, S.: Guide to Elliptic Curve Cryptography. Springer (2004). <https://doi.org/10.1007/b97644>
 10. Kannwischer, M.J., Rijneveld, J., Schwabe, P., Stoffelen, K.: pqm4: Testing and benchmarking NIST PQC on ARM Cortex-M4. IACR ePrint 2019/844 (2019), GitHub commit `cc2c1b992b60`
 11. Langley, A., Hamburg, M., Turner, S.: Elliptic curves for security. Tech. Rep. RFC 7748, IETF (2016)
 12. Lenngren, E.: X25519 for ARM Cortex-M4. GitHub repository `Emill/X25519-Cortex-M4` (2017)
 13. NIST: Module-lattice-based key-encapsulation mechanism standard. Tech. Rep. FIPS 203, NIST (2024)
 14. SLOTHY Optimizer Project: pqmx: Post-quantum cryptography on Arm Cortex-M. GitHub repository `slothy-optimizer/pqmx` (2026), commit `89465b9756cd`